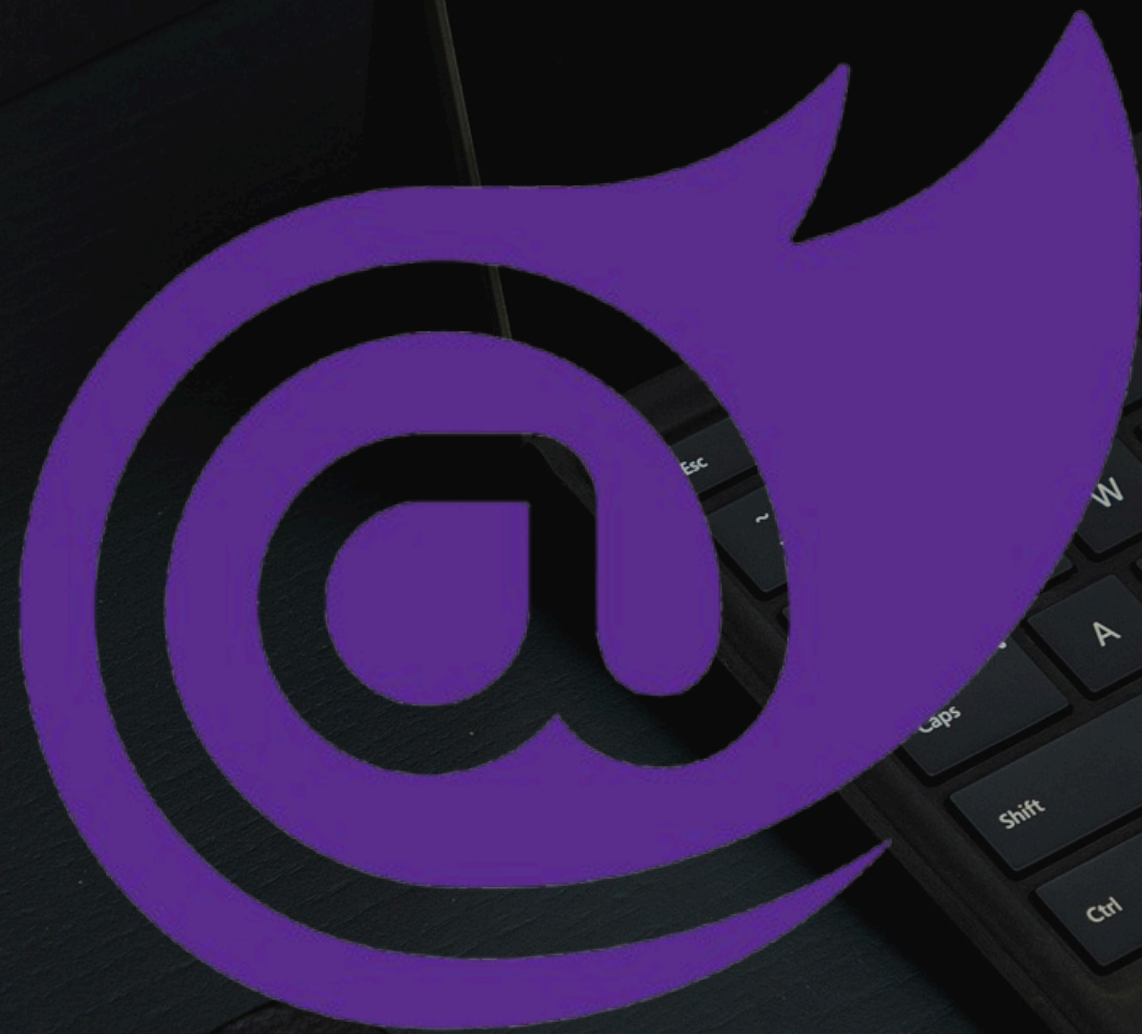


BLAZOR



UN FRAMEWORK .NET POUR LE WEB



AVANT BLAZOR : RAZOR PAGES ET MVC

Avant Blazor, on utilisait principalement Razor Pages et le modèle MVC (Model-View-Controller).

RAZOR EST UN MODELE DE TEMPLATE MELANT C# ET
HTML

MVC EST UN MODÈLE D'ARCHITECTURE (OU DESIGN
PATTERN)

MODEL POUR LES DONNÉES

VIEW POUR L'AFFICHAGE

CONTROLLER POUR LA LOGIQUE DE TRAITEMENT

Blazor

2019 : ARRIVÉE DE BLAZOR

POURQUOI BLAZOR ?

BLAZOR = BROWSER + RAZOR



Blazor : deux modèles

Blazor Server

LE CODE S'EFFECTUE COTÉ SERVEUR



CHARGEMENT RAPIDE
PAS DE CODE À COMPIÉ



NÉCESSITE UNE CONNEXION
PERMANENTE AU SERVEUR

Blazor Web Assembly (WASM)

LE CODE S'EFFECTUE COTÉ NAVIGATEUR



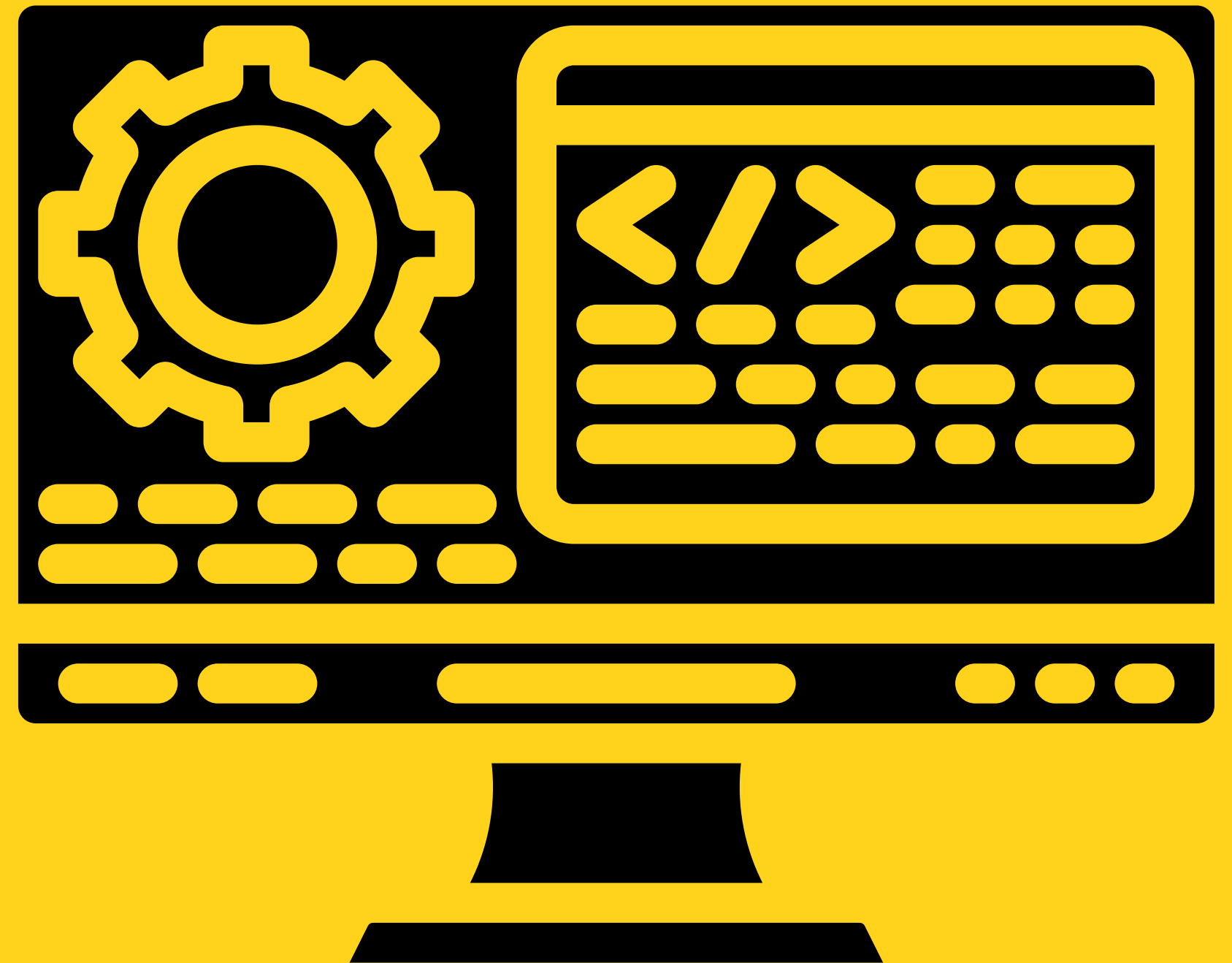
INDÉPENDANCE VIS À VIS DU SERVEUR



TEMPS DE CHARGEMENTS PLUS LONG

.NET 8 L'UNIFICATION

DEPUIS .NET 8, MICROSOFT A INTRODUIT
UNE APPROCHE APPELÉE BLAZOR
UNITED, QUI PERMET DE COMBINER LES
DEUX MODES (SERVER ET WASM) DANS
UNE MÊME APPLICATION



COMPOSANTS RAZOR

Blazor repose sur des composants Razor : ce sont des fichiers .razor qui contiennent un mélange de HTML et de C#, réutilisables et modulaires.

Chaque composant peut contenir de la logique C#, des événements, du binding de données, etc.

```
@foreach (var todo in Todos)
{
    <li class="list-group-item position-relative">
        <input class="big-checkbox" type="checkbox"
            checked="@todo.IsDone"
            @onchange="async e =>
            { todo.IsDone = ((ChangeEventArgs)e).Value as bool? ?? false;
              await SaveTodosAsync();}" />
        <span class="fs-4">@todo.Title</span>
        <button class="btn btn-danger position-absolute top-0 end-0" @onclick="() => RemoveTask(todo)">X</button>
        <br />
        <span>Created : @todo.DateCreated.ToString("yyyy/MM/dd")</span>
    </li>
}
</ul>

<EditForm Model="@this" OnValidSubmit="AddTask">
    <InputText class="form-control text-center mx-auto w-50 mt-5 fs-4" @bind-Value="text" />
    <button type="submit" class="btn btn-primary fs-4 px-3 m-3">Add a task to do</button>
</EditForm>
```

Blazor et Entity Framework (EF Core)

Blazor Server

- ON PEUT UTILISER EF CORE DIRECTEMENT AVEC UN DBCONTEXT
- CELA PERMET D'ACCÉDER À LA BASE DE DONNÉES FACILEMENT SANS ÉCRIRE DE REQUÊTES SQL MANUELLES

Blazor WASM

- EF CORE NE FONCTIONNE PAS DIRECTEMENT DANS LE NAVIGATEUR
- IL FAUT PASSER PAR UNE API ASP.NET CORE (VIA HTTP) POUR FAIRE LE LIEN ENTRE :
LE BACK-END
LE FRONT-END

CONCLUSION

Blazor est une solution moderne, puissante et cohérente avec l'écosystème .NET

Il permet aux développeurs C# de construire des applications web interactives sans avoir besoin de JavaScript

Selon le besoin, on peut choisir entre Blazor Server, Blazor WASM, ou Blazor Hybrid

Et grâce à Entity Framework, la gestion de la base de données reste fluide et en C#